

عملیات نجات گیت

- سطح: ساده

- طراح: رامتین بدری

سپاوش و دوستان برنامه‌نویسش روزی تصمیم گرفتند یک شرکت استارت‌آپی راه بندازند و دور هم جمع شدند تا یک پروژه خفن بزنند. همه‌چی خوب پیش می‌رفت تا وقتی که هرکی برای خودش شروع کرد فایل‌ها رو تغییر دادن.

یکی روی لاگین کار می‌کرد، یکی پرداخت رو دستکاری می‌کرد، یکی هم بدون خبر فایل README رو تغییر می‌داد! نتیجه؟ پروژه تبدیل شد به یک فاجعه تمام‌عیار.

کامیت‌ها قاطی شده بودند، برنچ‌ها بی‌نظم بودند، و وقتی می‌خواستند کدها رو merge کنند... به conflict‌های سرسام‌آوری برخوردند! حالا تیم فهمیده بودند که بدون Git اصولی، هیچ شانس برای ادامه ندارن. برای همین از شما خواستند که به نجات پروژه آنها بیایید و نشان بدهید که چطور با Git حرفه‌ای کار کنند.

در این بخش گام‌ها و مراحل که باید انجام دهید، به صورت زیر است:

مراحل

۱. یک پوشه با نام `project` بسازید و داخل آن با دستور `git init` یک repository ایجاد کنید.
۲. نام خود را تنظیم کنید (به نام دلخواه).
۳. با استفاده از دستور `touch` سه فایل `README.md`، `logs.txt` و `structure.txt` را ایجاد کنید و آن‌ها را با پیام `"initial commit"` کامیت کنید.
۴. در فایل `README.md` با استفاده از دستور `echo` یک توضیح کوتاه درباره پروژه بنویسید و آن را به staging area اضافه کرده و با پیام `"update README"` کامیت کنید.
۵. یک branch جدید با نام `feature-login` ایجاد کرده و به آن سویچ کنید.
۶. در این branch، فایل به نام `login.txt` ایجاد کنید و داخل آن یک متن دلخواه بنویسید. سپس آن را با پیام `"add login feature"` کامیت کنید.
۷. به branch اصلی (`main` یا `master`) برگردید و یک branch جدید با نام `feature-payment` ایجاد کنید.

۸. در این branch، فایلی به نام payment.txt ایجاد کرده و آن را با پیام "add payment feature" کامیت کنید.

۹. به branch اصلی برگردید و ابتدا branch feature-login را merge کنید.

۱۰. سپس branch feature-payment را merge کنید.

۱۱. در فایل README.md در branch اصلی یک خط جدید اضافه کنید با متن: "Project supports login and payment" و commit کنید.

۱۲. حالا به branch feature-login برگردید و همان فایل README.md را تغییر دهید (یک متن متفاوت اضافه کنید) و commit کنید.

۱۳. مجدداً به branch اصلی برگردید و تلاش کنید branch feature-login را merge کنید.

• در این مرحله یک **conflict** رخ خواهد داد.

۱۴. با استفاده از دستور nano وارد README.md شوید و conflict را به صورت دستی حل کنید به طوری که هر دو تغییر در فایل باقی بمانند. سپس فایل را add و با پیام "resolve conflict" کامیت کنید.

۱۵. با استفاده از دستور مناسب، تاریخچه commitها را به صورت خلاصه (oneline) و همراه با گراف (Git Graph) نمایش دهید.

نکته

شما فقط لازم است تمام عملیات را در ریپازیتوری لوکال خودتان انجام دهید و نیازی به پوش کردن کامیتها به ریپازیتوری remote در گیت‌هاب نیست.

چیزی که باید تحویل دهید

همه عملیات‌های گفته شده را در Git Bash انجام دهید. در نهایت باید یک فایل zip حاوی repository پروژه (دایرکتوری project) و عکس Gitgraph مربوط به مرحله ۱۵ را ارسال کنید.

بازگشت به گذشته با گیت

- سطح: متوسط

- طراح: آرتین عضدی‌فر

شما در یک شرکت به عنوان برنامه نویس مشغول به کار روی یک پروژه هستید و برای مدیریت پروژه از ورژن کنترلر git استفاده می‌کنید. در دنیای توسعه نرم‌افزار، اشتباه کردن اجتناب‌ناپذیر است. گاهی یک کامیت اشتباه می‌زنیم، گاهی فایلی را حذف می‌کنیم که بعداً به آن نیاز پیدا می‌کنیم، و گاهی تغییرات ناقصی داریم که نمی‌خواهیم آن‌ها را کامیت کنیم اما باید موقتاً کنار بگذاریم تا یک مشکل اضطراری را حل کنیم. گیت ابزارهای قدرتمندی برای برگشت به عقب در اختیار ما می‌گذارد: git restore برای بازیابی فایل‌ها از کامیت‌های قبلی، git reset برای حذف کامیت‌های اخیر (با حفظ یا عدم حفظ تغییرات) شما همیشه می‌توانید به کمک گیت اشتباهات پیش آمده حین کار را رفع کنید و تقریباً هیچوقت مجبور به شروع از اول نشید.

۱. یک دایرکتوری جدید بسازید و گیت را در آنجا راه‌اندازی کنید.

۲. دو فایل زیر را بسازید:

۱. فایل app.txt و در آن version 1 بنویسید.

۲. فایل config.txt و در آن timeout=10 بنویسید.

تغییرات را با پیام initial کامیت کنید.

۳. رئیس به شما پیام می‌دهد که به دلیل تغییرات در ساختار پروژه دیگر نیازی به فایل config نیست و می‌توانید آن را حذف کنید. تغییر را اعمال کنید و با پیام دلخواه کامیت کنید.

۴. یک ویژگی کوچک جدید به برنامه اضافه می‌شود.

به فایل app.txt در یک خط جدید new feature را اضافه کنید و تغییرات را کامیت کنید.

۵. یک باگ کوچک را در app.txt رفع می‌کنید؛ در انتهای فایل bug fix را اضافه و کامیت کنید

۶. قرار است یک ویژگی بزرگ جدید به برنامه اضافه شود پس یک شاخه جدید با نام feature-x بسازید و

وارد آن شوید.

یک فایل جدید با نام `temp.txt` بسازید و در آن بنویسید `temporary edits`.

هنوز تغییرات را تکمیل و کامیت نکرده‌اید که رئیس‌تان دوباره به شما پیام می‌دهد و می‌گوید:

«به دلایل فنی باید به ساختار قبلی برگردیم. لازم است فایل `config.txt` را بازگردانی و ویژگی جدیدی که به `app.txt` در مرحله ۴ اضافه شد را حذف کنی. البته باگی که رفع کرده‌ای نباید برگردد.»

از آنجایی که تغییرات شما روی این شاخه هنوز ناقص است و نمی‌خواهید یک سری تغییرات ناقص را کامیت کنید، به کمک دستور `stash` تغییرات را در پشته `stash` گیت ذخیره کنید و به شاخه `main` بروید

۷. به کمک دستور `log`، هش آخرین کامیتی که در آن فایل `config` وجود داشت را پیدا کنید و به کمک دستور `restore` آن فایل را بازگردانی کنید. همچنین ویژگی جدید را از `app.txt` حذف کنید (بدون از دست دادن `bug fix`). سپس با پیام `fixed some problems` کامیت کنید.

۸. رئیس از شما می‌خواهد سریعاً یک فایل جدید به نام `developers.txt` بسازید و در آن نام خود را به عنوان برنامه‌نویس شرکت بنویسید.

شما تغییرات را انجام می‌دهید و کامیت می‌کنید.

۹. هنوز سر کار اصلی‌تان برنگشته‌اید که رئیس که از کم‌خوابی مفرط توانایی تصمیم‌گیری خود را از دست داده تماس می‌گیرد و می‌گوید از تغییرات مرحله قبل پشیمان شده و می‌خواهد به طور کامل حذفش کند.

به کمک دستور `reset` کامیت آخر را حذف کنید و فایل را نیز حذف کنید.

نکته: شما حواستان هست که ممکن است رئیس دوباره از کارش پشیمان شود. پس برای این کار از `reset --soft` استفاده می‌کنید تا تغییرات در Stage باقی بماند و اگر دوباره خواست فایل را برگردانید، بدون نیاز به `Add` کردن مجدد قابل کامیت کردن باشد.

سپس فایل را `unstage` کنید و حذف کنید.

۱۰. بالاخره به شاخه `feature-x` بازگردید و با کمک بازگردانی از پشته `stash` تغییرات ناقص‌تان را بازیابی کنید.

در انتهای فایل `temp.txt` یک `done` اضافه کنید و تغییرات را کامیت کنید. سپس با شاخه اصلی (`main`) ادغام (`merge`) کنید.

۱۱. با استفاده از دستور مناسب، تاریخچه commit ها را به صورت خلاصه (oneline) و همراه با گراف (Git Graph) نمایش دهید.

آنچه باید آپلود کنید

همه عملیات های گفته شده را در Git Bash انجام دهید. در نهایت باید یک فایل zip حاوی repository پروژه (دایرکتوری project) و عکس Gitgraph مرحله آخر ارسال کنید.

سیستم مدیریت آژانس اصلی

- سطح: متوسط

- طراح: امیرحسین علی پور

مدیر یک آژانس مسافرتی قصد دارد سیستمی ساده برای رزرو تورهای مسافرتی و مدیریت بلیت‌های مشتریان طراحی کند. در این سیستم، تورهای مختلفی شامل حمل‌ونقل و اقامت در هتل وجود دارد و هر مسافر (عادی یا VIP) می‌تواند یک سفر را رزرو کند. شما باید با استفاده از مفاهیم **برنامه نویسی شیء‌گرا (OOP)** این سیستم را پیاده‌سازی کنید.

[برای دانلود فایل اولیه پروژه اینجا کلیک کنید](#)

کلاس‌های مورد نیاز

۱. کلاس Hotel

نماینگر محل اقامت مسافر است.

- ویژگی‌ها (private):

- نام هتل: `String hotelName`

- تعداد ستاره (از نوع `enum`): `HotelStar stars`

- قیمت هر شب اقامت: `int pricePerNight`

- متدها:

- **سازنده:** مقادیر اولیه را دریافت و ست می‌کند.

- هزینه کل اقامت را بر اساس تعداد شب‌ها محاسبه می‌کند. `calculateStayCost(int`

- `nights)`

- متدهای `Getter` و `Setter` برای تمامی ویژگی‌ها.

۲. کلاس Passenger

نماینگر یک مسافر در سیستم است.

• ویژگی‌ها (private):

- نام مسافر: `String name`
- وضعیت مسافر (از نوع `enum`): `PassengerStatus status` (عادی یا VIP)
- موجودی حساب: `int balance`

• متدها:

- **سازنده:** مقادیر اولیه را دریافت و ست می‌کند.
- `canAfford(int amount)`: بررسی می‌کند آیا موجودی مسافر برای پرداخت مبلغ مورد نظر کافی است یا خیر.
- `pay(int amount)`: در صورت کافی بودن موجودی، مبلغ را از حساب مسافر کسر می‌کند.
- متدهای `Getter` و `Setter` برای تمامی ویژگی‌ها.

۳. کلاس `Trip`

نماینگر یک پکیج مسافرتی است.

• ویژگی‌ها (private):

- شناسه سفر: `String tripId`
- نوع وسیله نقلیه (از نوع `enum`): `TransportType transport` (پرواز، قطار، اتوبوس)
- قیمت پایه بلیت: `int ticketBasePrice`
- هتل محل اقامت: `Hotel hotel`

• متدها:

- **سازنده:** مقادیر اولیه را دریافت و ست می‌کند.
- `getTotalTripCost(int nights)`: مجموع قیمت پایه بلیت و هزینه اقامت در هتل را برمی‌گرداند.
- متدهای `Getter` و `Setter` برای تمامی ویژگی‌ها.

۴. کلاس `Ticket`

نماینگر یک بلیت صادر شده برای یک مسافر مشخص است.

• ویژگی‌ها (private):

- مسافر: `Passenger passenger`

◦ سفر: Trip trip

◦ تعداد شب‌های اقامت: int nights

• متدها:

- **سازنده:** مقادیر اولیه را دریافت و ست می‌کند.
- calculateFinalPrice(): قیمت نهایی بلیت را به صورت زیر محاسبه می‌کند:
 ۱. مبلغ کل سفر (بلیت + هتل) محاسبه شود.
 ۲. اگر مسافر VIP باشد، ۱۵٪ تخفیف روی مبلغ اعمال شود.
 ۳. در نهایت ۹٪ مالیات به مبلغ اضافه شود.
 ۴. خروجی باید به صورت عدد صحیح (int) باشد (بخش اعشاری حذف شود).

۵. کلاس TravelAgency

مدیریت رزروها و آمار آژانس را بر عهده دارد.

• ویژگی‌ها (private):

- نام آژانس: String agencyName
- آرایه‌ای از بلیت‌ها: Ticket[] bookings (ظرفیت آرایه ۱۰۰ در نظر گرفته شود).
- تعداد رزروهای ثبت شده: int bookingCount

• متدها:

- bookTrip(Passenger p, Trip t, int nights):
 - ابتدا یک بلیت موقت ایجاد کرده و قیمت نهایی را محاسبه می‌کند.
 - اگر مسافر موجودی کافی داشت و ظرفیت آژانس پر نبود، مبلغ را کسر، بلیت را در آرایه ذخیره و true برمی‌گرداند.
- getTotalRevenue(): مجموع مبالغ دریافتی از تمام بلیت‌های صادر شده را محاسبه می‌کند.
- countTripsByTransport(TransportType type): تعداد سفرهایی که با یک نوع وسیله نقلیه خاص انجام شده‌اند را برمی‌گرداند.

BankX

سطح: متوسط

طراح: آرین صیدی

ایلان ماسک در حال برنامه‌ریزی برای اولین کلونی انسانی در مریخ است. او به خوبی می‌داند که سیستم‌های بانکی سنتی زمین، با آن بوروکراسی‌گند و باگ‌های امنیتی، در فضا دوام نمی‌آورند. ایلان به دنبال یک مهندس نرم‌افزار نترس می‌گردد تا اولین بانک خصوصی مریخ را طراحی کند.

[برای دانلود فایل اولیه پروژه اینجا کلیک کنید](#)

جزئیات پیاده‌سازی

شما باید سیستم بانکی را با رعایت دقیق معماری زیر پیاده‌سازی کنید:

۱. کلاس/Enum AccountTier (سطح حساب)

این بخش نشان‌دهنده نوع حساب کاربر است و ضرایب کارمزد و سود ماهانه را مشخص می‌کند. **این کلاس از قبل برای شما پیاده‌سازی شده است و نیازی به تغییر یا نوشتن کدهای آن ندارید.** صرفاً باید از مقادیر و متدهای آن در محاسبات خود استفاده کنید. مقادیر این ضرایب دقیقاً به شرح زیر پیاده‌سازی شده‌اند:

- REGULAR:

ضریب کارمزد: 0.005 | ضریب سود: 0.05

- SILVER:

ضریب کارمزد: 0.003 | ضریب سود: 0.10

- GOLD:

ضریب کارمزد: 0.001 | ضریب سود: 0.15

- PLATINUM:

ضریب کارمزد: 0.0 | ضریب سود: 0.20

(نکته: مبلغ کارمزد در هر تراکنش برابر است با مبلغ انتقال * ضریب کارمزد حساب مبدأ. همچنین مبلغ سود ماهانه برابر است با کمترین موجودی ماه * ضریب سود)

۲. کلاس BankAccount (حساب بانکی)

این کلاس وظیفه نگهداری و مدیریت دارایی هر شخص را دارد.

فیلدها:

- balance : موجودی حساب (از نوع double)
- accountNumber : شماره حساب
- ownerName : نام صاحب اکانت
- accountType : نوع/سطح اکانت (کلاس AccountTier)
- isBlocked : وضعیت مسدودی حساب

محدودیت‌های اصلی حساب:

۱. اگر اکانت مسدود (Blocked) باشد، **نمی‌توان** به آن واریز کرد یا از آن برداشتی انجام داد.
۲. هر کاربر در یک روز **بیشتر از ۱۰۰۰ واحد** نمی‌تواند از حساب خود برداشت کند.

متدهای الزامی:

- **متد withdraw(double amount)** : وظیفه برداشت (کسر) پول از حساب را دارد.
 - در صورت درخواست مبلغی بیشتر از موجودی (با احتساب کارمزد)، پیغام زیر چاپ شود:


```
1 | [FAILED] Insufficient balance for account {accountNumber}. Require
```
 - اگر محدودیت برداشت روزانه (۱۰۰۰ واحد) نقض شود:


```
1 | [FAILED] Daily withdrawal limit (1000) exceeded for account {accou
```
 - در صورت موفقیت‌آمیز بودن برداشت:

1 | [SUCCESS] Withdrawal: {amount} | Fee: {fee} | Account: {accountNum

• **متد deposit(double amount) :** وظیفه واریز پول به حساب را دارد.

◦ در صورت موفقیت‌آمیز بودن واریز:

1 | [SUCCESS] Deposit: {amount} | Account: {accountNumber} | New Bal

۳. کلاس Bank (هسته مرکزی)

فیلدها:

- bankAccounts : آرایه‌ای برای نگهداری اکانت‌های ثبت شده در بانک.
- profit : فیلد نگهداری سود کل بانک (حاصل از جمع‌آوری کارمزد تراکنش‌ها).

متدهای الزامی:

• **متد addBankAccount(BankAccount bankAccount) :** وظیفه ثبت اکانت جدید را دارد.

◦ **ظرفیت آرایه:** ظرفیت اولیه آرایه ۵ می‌باشد. در صورت پر شدن، ظرفیت باید به طور خودکار دو برابر شود.

◦ شماره حساب باید یکتا باشد. در صورت تکراری بودن شماره حساب:

1 | [ERROR] Account creation failed: Account number {accountNumber} al

◦ در صورت ثبت موفقیت‌آمیز اکانت:

1 | [INFO] New account registered: {accountNumber} for {ownerName}

• **متد processTransfer(TransferRequest transferRequest) :** وظیفه پردازش تراکنش‌ها

را بر عهده دارد. (درصد کارمزد بر اساس نوع اکانت مبدأ مشخص می‌شود).

◦ جلوگیری از تراکنش تکراری (اگر قبلاً پردازش شده باشد):

1 | [SKIP] Transaction {RequestID} was already processed.

◦ اگر مبلغ تراکنش غیرمجاز (صفر یا منفی) باشد:

1 | [FAILED] Transaction {RequestID}: Invalid amount (≤ 0).

○ مبدأ و مقصد نمی‌توانند یکسان باشند:

1 | [FAILED] Transaction {RequestID}: Source and destination are the s

○ در صورتی که حسابی پیدا نشود یا مسدود باشد، بسته به نوع خطا **تنها یکی** از پیام‌های زیر چاپ شود:

1 | [FAILED] Transaction {RequestID}: Source not found

1 | [FAILED] Transaction {RequestID}: Destination not found

1 | [FAILED] Transaction {RequestID}: Source is blocked

1 | [FAILED] Transaction {RequestID}: Destination is blocked

○ اگر تراکنش با موفقیت انجام شد:

1 | [SUCCESS] Transaction {RequestID} completed.

○ اگر تراکنش به هر دلیل دیگری موفقیت‌آمیز نبود:

1 | [FAILED] Transaction {RequestID} could not be completed.

• **متد `applyMonthlyInterest()` :** این متد در پایان ماه اجرا می‌شود.

○ **شرط سوددهی:** مبلغ سود با توجه به **کمترین موجودی مانده در اکانت در طول ماه** محاسبه و واریز می‌شود. حساب‌هایی که کمترین موجودی آن‌ها در طول ماه کمتر از ۱۰۰ واحد بوده، سودی دریافت نمی‌کنند. حساب‌های مسدود نیز مشمول سود نمی‌شوند.

○ پس از واریز سود برای هر حساب واجد شرایط:

1 | Interest of {amount} applied to Account {AccountNumber}. New Balar

• **متد** `setBlock(String accountNumber, boolean blocked)` : این متد وضعیت بلاک

بودن یا نبودن اکانت را تغییر می‌دهد.

○ اگر اکانت وجود داشت، بسته به وضعیت جدید پیغام متناسب چاپ شود:

```
1 | Account {accountNumber} LOCKED
```

یا:

```
1 | Account {accountNumber} UNLOCKED
```

○ اگر اکانت پیدا نشود:

```
1 | Security update failed: Account {accountNumber} not found.
```

⚠ **توجه:** تمامی مقادیر عددی مربوط به پول (مثل موجودی، کارمزد، مبلغ تراکنش و سود) در خروجی‌های چاپی باید دقیقاً تا دو رقم اعشار نمایش داده شوند.

⚠ تنها کلاس `Bank` صلاحیت تغییر در اکانت‌های ثبت شده را دارد و هرگونه تغییر خارج از کلاس بانک نباید در سیستم ثبت شود. در استفاده از **رفرنس‌ها (پوینترها)** دقت فرمایید.

مثال

ورودی نمونه ۱:

```
1 | Bank marsBank = new Bank();
2 | BankAccount acc1 = new BankAccount("A-100", "Elon", AccountTier.PLATINUM, 500
3 | BankAccount acc2 = new BankAccount("A-200", "Mark", AccountTier.GOLD, 2000.0,
4 | marsBank.addBankAccount(acc1);
5 | marsBank.addBankAccount(acc2);
6 |
7 | TransferRequest t1 = new TransferRequest(1, "A-100", "A-200", 500.0);
8 | marsBank.processTransfer(t1);
9 | marsBank.processTransfer(t1);
10 |
11 | marsBank.setBlock("A-200", true);
12 |
13 |
```

```
TransferRequest t2 = new TransferRequest(2, "A-200", "A-100", 100.0);
marsBank.processTransfer(t2);

marsBank.applyMonthlyInterest();
```

خروجی نمونه ۱:

```
1 [INFO] New account registered: A-100 for Elon
2 [INFO] New account registered: A-200 for Mark
3 [SUCCESS] Withdrawal: 500.00 | Fee: 0.00 | Account: A-100 | New Balance: 4500
4 [SUCCESS] Deposit: 500.00 | Account: A-200 | New Balance: 2500.00
5 [SUCCESS] Transaction 1 completed.
6 [SKIP] Transaction 1 was already processed.
7 Account A-200 LOCKED
8 [FAILED] Transaction 2: Source is blocked
9 [SUCCESS] Deposit: 900.00 | Account: A-100 | New Balance: 5400.00
10 Interest of 900.00 applied to Account A-100. New Balance: 5400.00
```

پست آفیس

• سطح: سخت

• طراح: دانیال یاقوتی

تبریک میگم! شما به عنوان توسعه دهنده سیستم مدیریت اداره پست شهری (PostOffice) انتخاب شدید.



ساختار فایل‌ها

```
fileName.zip
├─ PostOffice.java
├─ Postman.java
├─ Package.java
├─ Package_Status.java
├─ Letter.java
├─ Parcel.java
└─ Customer.java
```

برای دانلود فایل اولیه پروژه کلیک کنید 📦

جزئیات پروژه

کلاس Package_Status

این کلاس که وضعیت یک مرسوله پستی را مشخص می‌کند، از نوع Enum بوده و دارای سه وضعیت زیر می‌باشد:

۱. Waiting : در صف انتظار برای تحویل به پستی
۲. Processing : تحویل یافته به پستی و در حال ارسال
۳. Completed : تحویل داده شده

کلاس Package

ویژگی‌ها:

فیلد	نوع	توضیح
trackingCode	String	شناسه یکتا هر مرسوله
Status	Package_Status	وضعیت مرسوله
Distinction	String	آدرس مقصد
Area	int	عدد ناحیه پستی
Cost	double	هزینه مرسوله
Sender	Customer	فرستنده
Receiver	Customer	گیرنده

سازنده:


```
public Package(String trackingCode,
               Customer Sender,
               Customer Receiver,
               int Area,
               double Cost)
```

- وضعیت پیش‌فرض هر Package بعد از ساخته شدن باید روی حالت Waiting قرار بگیرد.
- مقصد پستی هر مرسوله باید از روی آدرس Receiver مشخص شود.
- نیازی به تغییر getter/setter ها نیست.

کلاس Parcel

هر Parcel فرزند کلاس Package است.

ویژگی‌های اضافی:

فیلد	نوع	توضیح
Weight	double	وزن بسته

سازنده:

```
public Parcel(String trackingCode,
               Customer Sender,
               Customer Receiver,
               int Area,
               double Cost,
               double Weight)
```

کلاس Letter

هر Letter فرزند کلاس Package است.

ویژگی‌های اضافی:

فیلد	نوع	توضیح
VIP	boolean	مشخص کننده فوری یا عادی بودن نامه

سازنده:

```
public Letter(String trackingCode,
              Customer Sender ,
              Customer Receiver,
              int Area,
              double Cost,
              boolean isVIP)
```

کلاس Postman

ویژگی‌ها:

فیلد	نوع	توضیح
id	String	شناسه پستی
workingArea	int	ناحیه کاری پستی
Box	Package[]	باکس مرسولات
LOAD	int	تعداد مرسولات در حال تحویل
DELIVERED	int	تعداد کل مرسولات تحویل داده شده

سازنده:

```
public Postman(String id, int workingArea, int boxSize)
```

- ظرفیت باکس پستی باید با مقدار ورودی تعیین گردد.

متدها:

1. متد addToBox:

```
public void addToBox(Package p)
```

- در صورت امکان مرسوله را به باکس پستی اضافه می‌کند.
- وضعیت مرسوله را به Processing را تغییر می‌دهد.
- LOAD پستی را آپدیت می‌کند.

2. متد deliver:

```
public void deliver()
```

- تمام مرسولات درون باکس پستی را تحویل می‌دهد.
- هر مرسوله را به صندوق received گیرنده آن اضافه می‌کند.
- وضعیت مرسولات تحویل داده شده را به Completed تغییر می‌دهد.
- مقدارهای LOAD و DELIVERED را به طور صحیح آپدیت می‌کند.
- در صورتی که بسته‌ای تحویل داده شد در انتها پیغام زیر را چاپ می‌کند:

```
Postman [Postman id] delivered: [Delivered Count] Box
```

- در صورتی که بسته‌ای برای تحویل وجود نداشت پیغام زیر چاپ شود:

```
Postman [Postman id] has nothing to deliver
```

3. متد isAvailable:

```
public boolean isAvailable()
```

- در صورتی که پستی قادر به تحویل گرفتن یک بسته دیگر از اداره‌پست باشد مقدار true و در غیر این صورت مقدار false را برمی‌گرداند.

کلاس Customer

ویژگی‌ها:

فیلد	نوع	توضیح
name	String	نام
address	String	آدرس
sent	Package[]	مرسولات فرستاده شده
received	Package[]	مرسولات دریافت شده
sentCounter	int	تعداد مرسولات ارسال شده
receivedCounter	int	تعداد مرسولات دریافت شده

نکته: تضمین می‌شود هر آدرس ثبت شده با فرمت City-Street-No باشد.

سازنده:

```
public Customer(String name, String Address)
```

- سایز اولیه برای آرایه های sent و received با مقدار 10 مقدار دهی شود.

متدها:

1. متد sentPackage:

```
public void sentPackage(Package p)
```

- مرسوله p را به صندوق ارسال شده ها اضافه می‌کند.
- در صورتی که صندوق پر شده بود، ظرفیت آن را 10 واحد اضافه و مجدد آپدیت می‌کند.

2. متد receivePackage:

```
public void receivePackage(Package p)
```

- مرسوله p را به صندوق دریافت شده ها اضافه می‌کند.
- در صورتی که صندوق پر شده بود، ظرفیت آن را 10 واحد اضافه و مجدد آپدیت می‌کند.

3. متد info:

```
public String info()
```

- یک رشته با فرمت زیر خروجی می‌دهد:

Name: [name] Address: [address] Sent: [sentCounter] Received: [receivedCounter]

کلاس PostOffice

این کلاس واحد اصلی مدیریت و کنترل اداره پست است.

ویژگی‌های الزامی:

فیلد	نوع	توضیح
activeCity	String	شهری که اداره پست در آن فعالیت می‌کند
BASE_COST	static double	مقدار هزینه پایه هر مرسوله
archive	Package[]	آرشیو همه مرسولات ثبت شده
postmans	Postman[]	آرایه شامل پستچی‌ها
waitingQueue	صف!	صف انتظار مرسولات برای تحویل به پستچی

سازنده:

```
public PostOffice(String Active_City)
```

- شهری که اداره پست در آن فعالیت می‌کند باید مشخص شود.
- مقدار BASE_COST به طور پیش فرض در این کلاس ثبت شده و نیازی به مقدار دهی یا تغییر آن نیست.

- مقدار اولیه آرایه های archive و postmans با مقدار 10 مقدار دهی شود.
- ظرفیت صف انتظار (در ادامه توضیح داده شده) باید دقیقاً برابر 10 باشد و قابلیت تغییر ندارد.

صف انتظار:

یک صف انتظار درواقع آرایه از Package ها است که اینطور باید مدیریت شود:

- اگر مرسوله‌ای به آن اضافه شد باید در انتهای مرسولات موجود قرار بگیرد (انتهای صف).
- اگر مرسوله‌ای از آن برداشته شد، مرسوله برداشت شده باید قدیمی ترین مرسوله‌ای باشد که در آن قرار دارد (ابتدای صف).
- اگر مرسوله ای از نوع Letter با فوریت VIP به آن اضافه شد، در ابتدای صف قرار بگیرد.
- این صف بیشتر از 10 واحد ظرفیت ندارد یعنی اگر 10 مرسوله ثبت شود و هیچ ارسالی رخ ندهد، مرسوله بعدی نباید قابلیت اضافه شدن داشته باشد!_

به بیانی دیگر، یک صف انتظار باید طوری مدیریت شود که عدالت برای ارسال مرسولات قدیمی تر یا مرسولات VIP برقرار شود. بنابراین پیاده‌سازی و مدیریت ارسال مرسولات ثبت شده به عهده خود شما بوده. توجه کنید که استفاده از بسته `java.util.Queue` مجاز نبوده و تنها مجاز به استفاده از آرایه ساده هستید.

▼ راهنمایی

برای پیاده‌سازی و مدیریت همچنین آرایه‌ای می‌توانید با ساخت یک کلاس `Private` در کلاس `PostOffice` با الگوی زیر یا فقط از الگوی آن استفاده کنید:

```

1 private class WaitingQueue {
2
3     private Package[] queue;
4
5     private final int MAX_CAPACITY;
6
7     private int add_head = 0;
8     private int take_head = 0;
9
10    private int counter = 0;
11
12    public WaitingQueue(int MAX_CAPACITY) {
13

```

```
        this.MAX_CAPACITY = MAX_CAPACITY;
        queue = new Package[MAX_CAPACITY];
    }

    public boolean isFull() {

        return counter == MAX_CAPACITY;
    }

    public boolean isEmpty() {

        return counter == 0;
    }

    public int size() {

        return counter;
    }

    public Package top() {

        // برگرداندن جلوترین مرسوله در صف
    }

    public void add(Package p) {

        // اضافه کردن یک مرسوله به انتهای صف
    }

    public void addToFront(Package p) {

        // اضافه کردن یک مرسوله به ابتدای صف
    }

    public Package take() {

        // حذف از صف و خروجی دادن مرسوله از ابتدای صف
    }

}
```

با مدیریت متغیرهای `take_head` (اشاره‌گر به موقعیت برداشت از آرایه) و `add_head` (اشاره‌گر به موقعیت قرار دادن در آرایه) و `counter` (تعداد عناصر موجود در صف) می‌توانید منطق مطلوب را پیاده‌سازی کنید.

متدها:

1. متد `registerNewPackage`:

```
public Package registerNewPackage(String type,
    Customer sender,
    Customer receiver,
    int Area,
    double weightOrVIP)
```

مدیریت منطق و چاپ پیغام در این متد باید به ترتیب زیر باشد:

۱. شهر مقصد مرسوله (گیرنده) با شهری که اداره پست در آن فعال است مقایسه شود؛ در صورت برابر نبودن پیغام زیر چاپ شود و `null` برگرداند:

```
Register Failed: Destination not equal to Active City
```

۲. بررسی شود صف انتظار ارسال مرسولات فضا داشته باشد، در صورت پر بودن تلاش شود با متد `sendAllPackage()` که در ادامه توضیح داده می‌شود برای ثبت بسته جدید فضا ایجاد کرده. در صورت شکست پیغام زیر چاپ شود و `null` برگرداند:

```
Register Failed: waitingQueue is full
```

۳. با توجه به تایپ دریافتی `type` در امضای متد یکی از دو شیء `Parcel` یا `Letter` ساخته می‌شود:

- اگر `type` برابر "Parcel" بود: پارامتر `weightOrVIP` وزن بسته در نظر گرفته می‌شود. هزینه‌ی مرسوله از فرمول زیر محاسبه شده و در نهایت یک شی از نوع `Parcel` برگردانده می‌شود:

$$Cost = baseCost + weight * 0.5$$

- اگر type برابر "Letter" بود: پارامتر weightOrVIP **اولویت مرسوله** در نظر گرفته می‌شود؛ تفسیر این پارامتر برای یک نامه به این شکل است:
- اگر weightOrVIP==1.0: مرسوله باید به ابتدای صف اضافه شده و مقدار فیلد VIP آن true باشد.
- اگر weightOrVIP==0.0: مرسوله باید به طور عادی به صف اضافه شده و مقدار فیلد VIP آن false باشد. در نهایت متد یک شیء از نوع Letter بر می‌گرداند با فرمول محاسبه هزینه به شرح زیر:

$$Cost = if(VIP) : baseCost + 7, else : baseCost$$

- در صورتی که تایپ ورودی برابر هیچکدام از دو مقدار معتبر بالا نبود پیغام زیر چاپ شود و null برمی‌گرداند:

Register Failed: Unknown Type

توجه:

- **کد رهگیری** هر مرسوله (trackingCode) باید یک کد یکتا با فرمت PKG-ddddd برای هر مرسوله باشد. که مقدار ddddd یک ترکیب 5 رقمی از اعداد است. وظیفه ساخت و مدیریت این کد ها با شماست. (تضمین می‌شود ساخت تعداد کدی بیش از 5! نیاز نیست)
- در انتهای این متد، در صورت ساخت موفق مرسوله، مرسوله باید به صندوق فرستنده متناظر ثبت شده و همچنین به آرشیو و صف انتظار مرسولات اضافه شده و در نهایت پیغام زیر چاپ شود:

New package registered

2. متد search:

```
public Package search(String trackingCode)
```

- مرسوله با کد رهگیری وارد شده جستجو شود، در صورتی که مرسوله متناظر پیدا شد آن را برگردانده و وضعیت مرسوله به شکل پیغام زیر چاپ شود:

Package Status: [Package Status]

- در صورت یافت نشدن، null برگردد و پیغام زیر چاپ شود:

Package Not Found

3. متد addPostman:

```
public void addPostman(Postman pm)
```

- یک پستی جدید به لیست پستی های اداره پست اضافه می کند. در صورتی آرایه مربوطه پر بود، 10 واحد به سائز آن اضافه کرده و مجدد آن را آپدیت کند.

4. متد sendPackage:

```
public boolean sendPackage ()
```

این متد سعی میکند تا اولین مرسوله در صف انتظار را به یک پستی تحویل دهد. مدیریت منطق و ترتیب چاپ پیغام به ترتیب زیر است:

۱. اگر صف انتظار خالی بود پیغام زیر چاپ شده و false برگرداند:

```
waitingQueue is empty
```

۲. در صورت خالی نبودن صف، سعی کند یک پستی مناسب از لیست پستی ها برای مرسوله مربوطه پیدا کند. انتخاب پستی مناسب برای یک بسته باید با شرایط زیر باشد:

- ناحیه کاری پستی workingArea با ناحیه پستی مرسوله Area یکسان باشد.
- پستی در دسترس باشد.
- از بین همه پستی های دارای شرایط بالا، باید پستی ای انتخاب شود که **کمترین** LOAD را داشته باشد.

در صورتی که پستی مناسب پیدا شد، مرسوله را به پستی تحویل دهد و مقدار true را برگرداند و

پیغام زیر چاپ شود:

Package [Package trackingCode] is taken by postman: [Selected postman id]

و در صورتی که پستی مناسب پیدا نشد مقدار false برگردانده و پیغام زیر چاپ شود:

No postman found

5. متد sendAllPackage:

```
public boolean sendAllPackage()
```

• اگر صف انتظار خالی بود false برمی‌گرداند و پیغام زیر را چاپ می‌کند:

waitingQueue is empty

• در غیر این صورت سعی می‌کند تمام مرسوله های پستی را به ترتیب موجود در صف به پستی‌ها تحویل دهد. با این تفاوت که اگر پستی مناسب برای مرسوله A پیدا نشد، مرسوله A را به انتهای صف دوباره اضافه می‌کند و تلاش می‌کند تا مرسوله ی بعدی را تحویل دهد. شماتیک الگوریتم ارسال در این متد به شکل زیر است:

```
queue: [A] [B] [C]
try send [A]
if success -> continue
else ->
1. take [A]
2. add [A]
-> queue: [B] [C] [A]
try send [B]
...
```

• اگر حداقل یک مرسوله تحویل داده شده بود مقدار true و در غیر این صورت false برگرداند و پیغام زیر در انتها چاپ شود:

Packages sent: [Success send count]

مثال

برای درک بهتر به مثال و خروجی زیر توجه کنید:

```
1 public class Main {
2
3     public static void main(String[] args) {
4
5         Customer A = new Customer("Shahrzad", "Shiraz-Nab-31");
6
7         Customer B = new Customer("Negar", "Tehran-Nasim-7");
8
9         Customer C = new Customer("Parynaz", "Shiraz-Yar-21");
10
11         Postman pm1 = new Postman("M1", 1, 1);
12
13         Postman pm2 = new Postman("M2", 2, 1);
14
15         PostOffice postOffice = new PostOffice("Shiraz");
16
17         postOffice.addPostman(pm1);
18
19         postOffice.addPostman(pm2);
20
21         Package p1 = postOffice.registerNewPackage("Packet", B,A, 1, 5.0); //
22
23         Package p2 = postOffice.registerNewPackage("Packet", A,B, 1, 5.0); //
24
25         Package p3 = postOffice.registerNewPackage("Parcel", B,A, 2, 5.0); //
26
27         Package p4 = postOffice.registerNewPackage("Parcel", C,A, 2, 7.5); //
28
29         Package p5 = postOffice.registerNewPackage("Letter", B,A, 1, 1); // A
30
31         System.out.println(p5);
32
33         postOffice.sendPackage(); // p5 -> pm1
34
35     }
```

```

        postOffice.search(p5.getTrackingCode()); // Status changed: Processin

        postOffice.sendPackage(); // p3 -> pm2

        postOffice.sendPackage(); // pm1 and pm2's boxes are full

        pm1.deliver();

        System.out.println("PM1 isAvailable: " + pm1.isAvailable()); //true

        System.out.println(B.info()); //sent: p3, p5

        System.out.println(A.info()); //received: p5

    }

}

```

خروجی:

```

1 | Register Failed: Unknown Type
2 | Register Failed: Destination not equal to Active City
3 | New package registered
4 | New package registered
5 | New package registered
6 | {Type: Letter, VIP: true, TrackingCode: PKG-00002, Destination: Shiraz-Nab-31
7 | Package PKG-00002 is taken by postman: M1
8 | Package Status: Processing
9 | Package PKG-00000 is taken by postman: M2
10 | No postman found
11 | Postman M1 delivered: 1 Box
12 | PM1 isAvailable: true
13 | Name: Negar Address: Tehran-Nasim-7 Sent: 2 Received: 0
14 | Name: Shahrzad Address: Shiraz-Nab-31 Sent: 0 Received: 1

```

هیاهول (امتیازی)

- سطح: امتیازی
- طراح: سعید خزاعی

در دنیایی پر از هیولاهای، **انجمن شکارچیان** (Guild) مرکزی برای هماهنگی شکارچیان، ثبت قراردادهای و مدیریت جوایز است.

وظیفه شما کمک به پیاده سازی این سیستم است.

ساختار فایلها

```
fileName.zip
├─ Hunter.java
├─ Monster.java
├─ Contract.java
├─ Guild.java
├─ BeastContract.java
├─ DragonContract.java
└─ RaidContract.java
```

برای دانلود فایل اولیه پروژه اینجا کلیک کنید

جزئیات پروژه

کلاس Monster

ویژگیها (protected):

فیلد	نوع	توضیح
monsterID	String	شناسه هیولا
name	String	نام هیولا

فیلد	نوع	توضیح
level	int	سطح (۱ تا ۵)
health	int	سلامت فعلی هیولا
bounty	int	جایزه (سکه)
isAlive	boolean	وضعیت زنده بودن

کانستراکتور:

```
public Monster(String monsterID, String name, int level, int health, int bounty)
```

- `isAlive` به صورت پیش فرض بر اساس `health` تعیین می شود.

متدها:

1. متد `takeDamage`:

```
public void takeDamage(int damage)
```

- آسیب وارد شده را از `health` کم می کند.
- اگر `health` به ۰ (یا کمتر) برسد:
 - توجه داشته باشید که باید موارد مورد نیاز را آپدیت کنید (همچنین دقت کنید کمترین مقدار `health` برای نگه داری ۰ است).
 - پیام زیر چاپ می شود:

```
[MonsterName] has been slain!
```

2. متد `getReward`:

```
public int getReward()
```

- مقدار `bounty` دریافت شده از کشتن هیولا را برمی گرداند. (در صورتی که هیولا قبلاً کشته شده

باشد، باید مقدار 0 را برگرداند.)

3. متد isDangerous:

```
public boolean isDangerous()
```

• بررسی می‌کند سطح هیولا برابر یا بیشتر از 4 است.

4. متد isAlive:

```
public boolean isAlive()
```

• زنده بودن هیولا را برمی‌گرداند.

5. گترها

کلاس Contract

ویژگی‌ها (protected):

فیلد	نوع	توضیح
contractID	String	شناسه قرارداد
title	String	عنوان قرارداد
targetMonster	Monster	هیولای هدف
reward	int	جایزه پایه (سکه)
requiredRank	int	رتبه مورد نیاز (۱ تا ۵)
isCompleted	boolean	وضعیت تکمیل قرارداد

کانستراکتور:

```
public Contract(String contractID, String title, Monster targetMonster, int reward,
```


- `isCompleted` به صورت پیش فرض `false` قرار می‌گیرد.

متدها:

1. متد `getDifficulty`:

```
public String getDifficulty()
```

- اگر `requiredRank` ۱ یا ۲ باشد: "Easy" را برمی‌گرداند.
- اگر ۳ باشد: "Medium" را برمی‌گرداند.
- اگر ۴ باشد: "Hard" را برمی‌گرداند.
- اگر ۵ باشد: "Legendary" را برمی‌گرداند.

2. متد `complete`:

```
public boolean complete()
```

- اگر قبلاً تکمیل شده: `false` برمی‌گرداند.
- در غیر این صورت: قرارداد را تکمیل کرده و `true` برمی‌گرداند.

3. متد `calculateFinalReward`:

```
public double calculateFinalReward(Hunter hunter)
```

- پاداش نهایی = $\text{reward} + (\text{hunter.getRank()} * 50.0)$

4. گترها

کلاس `BeastContract` (فرزند `Contract`)

ویژگی‌ها:

فیلد	نوع	توضیح

فیلد	نوع	توضیح
huntingZone	String	منطقه شکار

کانستراکتور:

```
public BeastContract(String contractID, String title, Monster targetMonster, int reward)
```

متدها:

1. متد `getDifficulty`:

```
public String getDifficulty()
```

- رشته زیر را برمی‌گرداند:

```
Beast - [Zone] - [Difficulty]
```

- اگر `huntingZone` برابر "DarkForest" باشد: رشته با پسوند "(+20% reward)" نمایش داده می‌شود.

2. متد `calculateFinalReward`:

```
public double calculateFinalReward(Hunter hunter)
```

- اگر `huntingZone` برابر "DarkForest" باشد: جایزه ۲۰٪ اضافه‌تر می‌شود.

کلاس `DragonContract` (فرزند `Contract`)

ویژگی‌ها:

فیلد	نوع	توضیح

فیلد	نوع	توضیح
hasArmor	boolean	وضعیت زره

کانستراکتور:

```
public DragonContract(String contractID, String title, Monster targetMonster, int r
```

متدها:

1. متد `calculateFinalReward`:

```
public double calculateFinalReward(Hunter hunter)
```

• اگر زره داشته باشد: جایزه ۵۰٪ اضافه‌تر می‌شود.

2. متد `getDifficulty`:

```
public String getDifficulty()
```

• اگر زره داشته باشد: "Legendary" برمی‌گرداند. (صرف‌نظر از `requiredRank`)

کلاس `RaidContract` (فرزند `Contract`)

ویژگی‌ها:

فیلد	نوع	توضیح
hunterCount	int	تعداد شکارچیان مورد نیاز
hunters	[]Hunter	آرایه شکارچیان (ظرفیت: ۱۰)
currentHunters	int	تعداد شکارچیان فعلی

کانستراکتور:

```
public RaidContract(String contractID, String title, Monster targetMonster, int rev
```

متدها:

1. متد addHunter:

```
public boolean addHunter(Hunter hunter)
```

- اگر تعداد شکارچیان به حد نصاب نرسیده: اضافه می‌کند و `true` برمی‌گرداند.
- در غیر این صورت: `false` برمی‌گرداند و پیام زیر را چاپ می‌کند.

```
Raid contract is full!
```

2. متد removeHunter:

```
public boolean removeHunter(Hunter hunter)
```

- شکارچی را حذف می‌کند. (اگر وجود دارد!)

3. متد isReadyToStart:

```
public boolean isReadyToStart()
```

- اگر تعداد شکارچیان کافی باشد: `true` برمی‌گرداند.

4. متد complete:

```
public boolean complete()
```

- اگر آماده شروع **نباشد**: `false` برمی‌گرداند.

◦ پیام:

```
Not enough hunters to start the raid!
```

5. متد calculateFinalReward:

```
public double calculateFinalReward(Hunter hunter)
```

• پاداش نهایی = (currentHunters * 20) + reward (پاداش همکاری تیمی)

کلاس Hunter

ویژگی‌ها (private):

فیلد	نوع	توضیح
hunterID	String	شناسه شکارچی
name	String	نام شکارچی
rank	int	رتبه (۱ تا ۵)
experience	int	تجربه کسب‌شده
contracts	[]Contract	آرایه‌ای از قراردادهای (ظرفیت پیش‌فرض: ۵)
contractCount	int	تعداد قراردادهای فعال
earnings	double	درآمد کل (سکه)

کانستراکتور:

```
public Hunter(String hunterID, String name)
```

- رتبه اولیه: ۱
- تجربه اولیه: ۰
- درآمد اولیه: ۰

متدها:

1. متد acceptContract:

```
public boolean acceptContract(Contract contract)
```

- اگر فصل شکار شروع نشده باشد: `false` برمی‌گرداند.

◦ پیام:

```
Action denied! Hunting season has not started.
```

- اگر ظرفیت قراردادها پر باشد: `false` برمی‌گرداند.

◦ پیام:

```
Cannot accept more contracts! Maximum reached.
```

- اگر رتبه شکارچی کمتر از رتبه قرارداد باشد: `false` برمی‌گرداند.

◦ پیام:

```
Hunter rank is too low for this contract!
```

- در صورت موفقیت: `true` برمی‌گرداند.

◦ پیام:

```
[HunterName] accepted contract: [ContractTitle]
```

2. متد `abandonContract`:

```
public boolean abandonContract(Contract contract)
```

- اگر شکارچی این قرارداد را نداشته باشد: `false` برمی‌گرداند.

◦ پیام:

```
[HunterName] is not assigned to this contract.
```

- در صورت موفقیت: `true` برمی‌گرداند.

◦ پیام:

[HunterName] abandoned contract: [ContractTitle]

3. متد completeContract:

```
public boolean completeContract(Contract contract)
```

- اگر قرارداد در لیست شکارچی نباشد: false برمی‌گرداند.

◦ پیام:

[HunterName] has not accepted this contract.

- در صورت موفقیت:

◦ پاداش به earnings اضافه شود.

◦ تجربه به experience اضافه شود. (بخش صحیح جایزه)

◦ اگر تجربه به ۱۰۰ برسد، رتبه ۱ واحد افزایش یابد (حداکثر رتبه: ۵) و تجربه **ریست** شود.

(ممکن است بیش از یک لول به سطح شکارچی اضافه شود.)

◦ پیام (با توجه به ارتقا رتبه، بخش (Rank Up!) اضافه می‌شود):

[HunterName] completed [ContractTitle] and earned [Reward] coins! (Rank

◦ یا:

[HunterName] completed [ContractTitle] and earned [Reward] coins.

4. متد getTotalEarnings:

```
public double getTotalEarnings()
```

- جمع کل درآمد از تمام قراردادهای تکمیل‌شده را برمی‌گرداند.

5. متد calculatePower:

```
public int calculatePower()
```

• قدرت شکارچی = $(\text{rank} * 10) + (\text{experience} / 10)$

6. متد `canHunt`:

```
public boolean canHunt(Monster monster)
```

• بررسی می‌کند رتبه شکارچی **کمتر** از سطح هیولا نباشد.

7. متد `getActiveContractCount`:

```
public int getActiveContractCount()
```

• تعداد قراردادهای فعال را برمی‌گرداند.

8. متد `clearHunterData`:

```
public void clearHunterData()
```

• تمام داده‌های شکارچی (به جز ID و name) پاک شود.

9. گترها

کلاس Guild

ویژگی‌ها (private):

فیلد	نوع	توضیح
guildName	String	نام انجمن
seasonStarted	boolean (static)	وضعیت فصل شکار
hunters	[]Hunter	آرایه شکارچیان

فیلد	نوع	توضیح
contracts	[]Contract	آرایه قراردادهای

کانستراکتور:

```
public Guild(String guildName, int maxHunters, int maxContracts)
```

متدها:

1. متد startSeason:

```
public static boolean startSeason()
```

- فصل شکار شروع می‌شود و مقدار true را برمی‌گرداند.
- پیام:

```
The hunting season has begun at The Guild!
```

- اگر از قبل فصل شکار شروع شده باشد، مقدار false را برمی‌گرداند.

2. متد endSeason:

```
public void endSeason()
```

- فصل شکار پایان می‌یابد.
- پیام:

```
The hunting season has ended. Total bounties paid to each hunter: [X]
```

- پاک‌سازی: داده‌های تمام شکارچیان و قراردادهای پاک شود.

3. متد registerHunter:

```
public boolean registerHunter(Hunter hunter)
```

- اگر فصل شروع نشده: false برمی گرداند.

◦ پیام:

```
Action denied! Hunting season has not started.
```

- اگر ظرفیت پر باشد: false برمی گرداند.

◦ پیام:

```
Guild is full! Cannot register more hunters.
```

- در غیر این صورت: true برمی گرداند.

4. متد `publishContract`:

```
public boolean publishContract(Contract contract)
```

- اگر فصل شروع نشده: false برمی گرداند.

◦ پیام:

```
Action denied! Hunting season has not started.
```

- اگر ظرفیت پر باشد: false برمی گرداند.

◦ پیام:

```
Guild cannot publish more contracts!
```

- در غیر این صورت: true برمی گرداند.

5. متد `assignContractToHunter`:

```
public boolean assignContractToHunter(Hunter hunter, Contract contract)
```

- اگر فصل شروع نشده: false برمی گرداند.

◦ پیام:

Action denied! Hunting season has not started.

- اگر شکارچی یا قرارداد در سیستم نباشد: false برمی‌گرداند.

◦ پیام:

Hunter or contract not found in guild records.

- در غیر این صورت: شکارچی قرارداد را قبول می‌کند.

6. متد `getTopHunter`:

```
public Hunter getTopHunter()
```

- شکارچی با بیشترین earnings را برمی‌گرداند.
- اگر شکارچی نباشد: null برمی‌گرداند.

7. متد `getHuntersByRank`:

```
public Hunter[] getHuntersByRank(int rank)
```

- آرایه‌ای از شکارچیان با رتبه مشخص را برمی‌گرداند.

8. متد `getActiveContracts`:

```
public int getActiveContracts()
```

- تعداد قراردادهای فعال (تکمیل نشده) را برمی‌گرداند.

9. متد `isSeasonStarted`:

```
public static boolean isSeasonStarted()
```

- وضعیت شروع فصل شکار را برمی‌گرداند.

10. متد getTotalBountiesPaid:

```
public double getTotalBountiesPaid()
```

- مجموع کل درآمد همه شکارچیان را برمی‌گرداند.

11. گترها

توجه کنید اسم گذاری متدهای گتر در تمامی کلاس ها باید به فرمت زیر باشد:

```
public <type> getFieldname()
```

- برای مثال برای فیلدی با نام testName از جنس استرینگ خواهیم داشت:

```
public String getTestName()
```